# Visitor Dashboard for mvcprogrammer.com

A plan for a private analytics dashboard over the site's own access logs and engagement beacon: what it shows, how it is built, how it is hosted and protected, and in what order the work lands. Prepared October 18, 2026.

## 1. Summary

The site already has the raw material: CloudFront writes a log line for every request to all four editions, and the engagement beacon adds page loads, section views, link clicks, and time on page. Today those logs are read by a command-line tool that prints one text report. The dashboard turns the same data into a page you can open on any device: headline numbers with trends, a daily chart, breakdowns by edition, referrer, screen, and country, the reading funnel through the page, and the clicks that matter most, with a date range picker, period-over-period comparison, and click-to-filter segments.

The recommendation keeps to the site's rule of adding nothing that is not earning its place:

- **A scheduled job, not a server.** A GitHub Actions workflow runs the existing log parser every six hours and writes small JSON files, one per day, into the site bucket. There is no database and no API.
- **A static page, like the four editions.** A React 19 and Vite app under `stats/` in the same repository, built and deployed by the same workflow, served at `stats.mvcprogrammer.com` through its own CloudFront distribution.
- **Private by design.** Amazon Cognito signs you in; a Lambda@Edge check at the CloudFront door lets nothing through without a valid session, including the data files. One user, passkey or authenticator app, nothing public.
- **Charts that respect the site's security policy.** Chart.js, which draws on canvas and sets no inline styles, so the dashboard runs under the same strict Content Security Policy as the site.
- **Under a dollar a month.** Every piece sits inside AWS free tiers at this traffic.

Five decisions are listed at the end for you to confirm or change; the plan is written so that each can be swapped without redoing the rest.

## 2. Goals and non-goals

**Goals**

- Answer, in under ten seconds of looking, how the site is doing this week against last week.
- Show the things the beacon was built for: which editions people choose, whether they click through to GitHub, how far down the page they read, and how long they stay.
- Let you choose any date range, compare it with the previous period, and narrow the whole view to one edition, screen class, color scheme, referrer, or country.
- Work on a phone as well as a laptop, in light and dark mode, with keyboard access throughout.
- Stay private, cheap, and dependency-light, and be a credible piece of engineering in its own right, since the repository is public.

**Non-goals**

- Real-time counters. Data refreshes every few hours; a "live now" number would need a different pipeline and is not worth it for a portfolio.
- Individual visitor timelines or any identifier that outlives a day. The dashboard reports aggregates.
- Replacing the command-line report. It stays as the quick check and as the shared parsing code.
- Multi-user access, roles, or sharing links. One sign-in, yours.

**Success criteria**

- The dashboard loads in under two seconds on a phone over cellular for a 30-day range.
- Every number on it can be reproduced from the JSON files by hand, and every definition is written down (section 6).
- Nothing about the dashboard changes the public site, its headers, or its cost profile.

## 3. What exists today

| Source | What it holds | Cadence | Where |
|---|---|---|---|
| CloudFront standard logs | One line per request: time, edge location, client IP, method, host, path, status, referrer, user agent, query string, cache result | Delivered within minutes to an hour, gzipped, several files a day | `s3://mvcprogrammer-cf-logs-<account>/{cf,blazor,angular,react}/` |
| Engagement beacon (`GET /ping`) | `load` (screen class, color scheme), `view` (section id), `click` (edition, GitHub, LinkedIn, Flickr, with target and place), `leave` (seconds visible) | Inline in the same logs, as query strings | Same files |
| Experience API requests | `/api/experience` and `/api/skills` hits, including skill-filter clicks (`?skill=`) | Same logs | Same files |
| Resume downloads | `/resume/*` hits with 200 or 206 | Same logs | Same files |
| `tools/cf-stats.mjs` | Downloads the files, drops scanners, groups requests into 30-minute visits, prints the text report | On demand | Repository |

Not available in the logs, and therefore out of scope for the first version: contact form submissions (they post to a Lambda function URL, not through CloudFront) and the resume download alerts (a separate Lambda). Both could be added later by having those Lambdas write a one-line JSON record to the same data prefix.

## 4. The recommendation at a glance

| Decision | Recommendation | Why | Alternative considered |
|---|---|---|---|
| Where the data is aggregated | A scheduled GitHub Actions workflow running the shared parser | Zero new infrastructure; reuses the OIDC deploy role, the parser, and the workflow conventions already in the repo; runs anywhere, not only on your Mac | A Lambda on an EventBridge schedule (cleaner "serverless" story, more to document); aggregation on demand in the browser (re-reads raw logs on every open, slow and wasteful) |
| Data format | One JSON file per day plus an index, served as static files | Any date range is a handful of small fetches; no API to secure or pay for; trivially cached; easy to inspect and test | A single growing file (fine for a year, then not); DynamoDB plus an API (overkill for one reader) |
| Front end | React 19 with Vite and TypeScript under `stats/` | The React edition already established the conventions, CSP handling, build, and deploy job to copy; you work in React daily | Angular (equally viable; React has the larger charting ecosystem); vanilla JavaScript (a dashboard with filters and charts is where a framework earns its place) |
| Charts | Chart.js 4 through react-chartjs-2 | Canvas-based, no inline styles, so it runs under the strict CSP; the most widely used charting library; small; good keyboard and screen-reader fallbacks | Recharts (SVG, inline styles on tooltips, blocked by the CSP); Apache ECharts (powerful, three times the size); uPlot (fastest, least documentation) |
| Hosting | `stats.mvcprogrammer.com`: the `stats/` prefix of the site bucket, its own CloudFront distribution and certificate, same security headers | The exact pattern used three times for the editions; one bucket, one deploy workflow | A local-only tool (zero cost, no phone access, no sign-in needed; kept as a fallback) |
| Sign-in | Amazon Cognito user pool with one user and a passkey or authenticator app, verified at the edge by Lambda@Edge | The standard AWS answer; no password of yours in any code or store; MFA for free; the data files are protected by the same check as the page | HTTP Basic auth through a CloudFront Function and a hashed secret in KeyValueStore (ships in an hour, browser prompt is ugly, no MFA, one shared secret) |
| Freshness | Every six hours, plus a manual run button | Matches how often a portfolio is worth checking; four runs a day cost nothing | Hourly (fine, just noisier in the Actions history) |
| Cost | Under one dollar a month | ACM, Cognito, Actions minutes on a public repo, S3 bytes, and Lambda@Edge invocations all sit inside free tiers or round to cents | |

## 5. Architecture

```
CloudFront (4 distributions)            GitHub Actions, every 6 hours
  access logs, gzipped W3C   -------->   stats.yml: node tools/stats-export.mjs
        |                                  reads the last N days of logs
        v                                  rolls them into per-day JSON
s3://mvcprogrammer-cf-logs-…/              uploads to s3://mvcprogrammer.com/stats/data/
   cf/  blazor/  angular/  react/               |
                                                v
                                s3://mvcprogrammer.com/stats/
                                  index.html, bundles/        (the dashboard app)
                                  data/index.json             (which days exist, schema version)
                                  data/days/2026-10-18.json   (one file per day)
                                                |
                                                v
                                CloudFront: stats.mvcprogrammer.com
                                  Origin Access Control to the stats/ prefix
                                  Lambda@Edge (viewer-request): valid Cognito session cookie, or
                                    redirect to the Cognito hosted sign-in page
                                  Same response headers policy as the site (CSP, HSTS, nosniff)
                                                |
                                                v
                                Browser: picks a date range, fetches the day files it needs,
                                  aggregates in memory, draws the charts
```

**Why aggregate per day and sum in the browser.** A day of this site is a few kilobytes of JSON. Thirty days is under 200 KB before compression. Summing thirty small objects in the browser is instant, and it means the date picker, the comparison, and the segment filters need nothing from a server. The only metric that does not sum across days is unique visitors; section 6 explains how the day files carry enough to compute it without identifiers.

**What stays the same.** The four editions, their distributions, headers, and logs are untouched. The dashboard reads logs; it never writes to anything the public site uses. The command-line report keeps working and shares the parsing code with the exporter.

## 6. Data model and metric definitions

### 6.1 Files

`stats/data/index.json`

| Field | Meaning |
|---|---|
| `schemaVersion` | Integer; the app refuses files from a newer schema and asks for a redeploy |
| `generatedAt` | When the last export ran, shown on the dashboard as "Updated 2 hours ago" |
| `days` | The list of day keys that have a file, so the picker can grey out gaps |
| `timeZone` | The zone days are cut on, `America/New_York`, so the chart labels and the report agree |

`stats/data/days/YYYY-MM-DD.json`, one per calendar day in the site's time zone:

| Field | Meaning |
|---|---|
| `schemaVersion`, `day`, `timeZone` | As in the index |

| Field | Meaning |
|---|---|
| `requests`, `scannerRequests` | Raw line count and the share dropped as bots or 4xx noise |
| `visits` | One record per visit that started on that day, in start order, with the fields below |

A visit record carries only what the dashboard needs, and leaves a field out when the visit has nothing for it:

| Field | Meaning |
|---|---|
| `token` | A short salted hash of the browser (IP plus user agent string); the same browser gets the same token on every day, so uniques can be counted across a range, but nothing can be reversed from it |
| `hour` | The local hour the visit started, for the time-of-day panel |
| `editions` | The editions it loaded a page on, in display order; a visit that moved between editions lists each |
| `referrer` | The external referrer host; absent for a direct visit |
| `country`, `edge` | The two-letter country of the CloudFront edge that served it (from a table of edge codes) and the edge code itself |
| `loads`, `screen`, `scheme` | From the load ping: how many times the script started, the screen class, and the color scheme; absent when the script never ran |
| `sections` | The sections reached, in page order |
| `seconds` | The largest leave value reported, in seconds visible |
| `clicks` | The edition, GitHub, and profile links followed, as `kind/target/place` |
| `apiCalls`, `skills`, `resumes` | Experience API calls, skill-filter clicks by skill, and resume downloads by edition tag |

Records rather than pre-summed counters, so any combination of filters is exact in the browser (unique visitors inside a segment included) and the files stay small at this site's traffic: about 200 bytes a visit, under 10 KB on a busy day, two to three megabytes a year. Should traffic ever grow past a few thousand visits a day, the exporter can add per-day aggregates beside the records without changing the app's math, and `schemaVersion` goes up. Files are regenerated for the last three days on every run, because CloudFront can deliver a log file hours after the requests in it. The owner's own IP address is excluded before anything is counted.

## 6.2 Definitions

These match the command-line tool where it already defines a term, and tighten it where it does not.

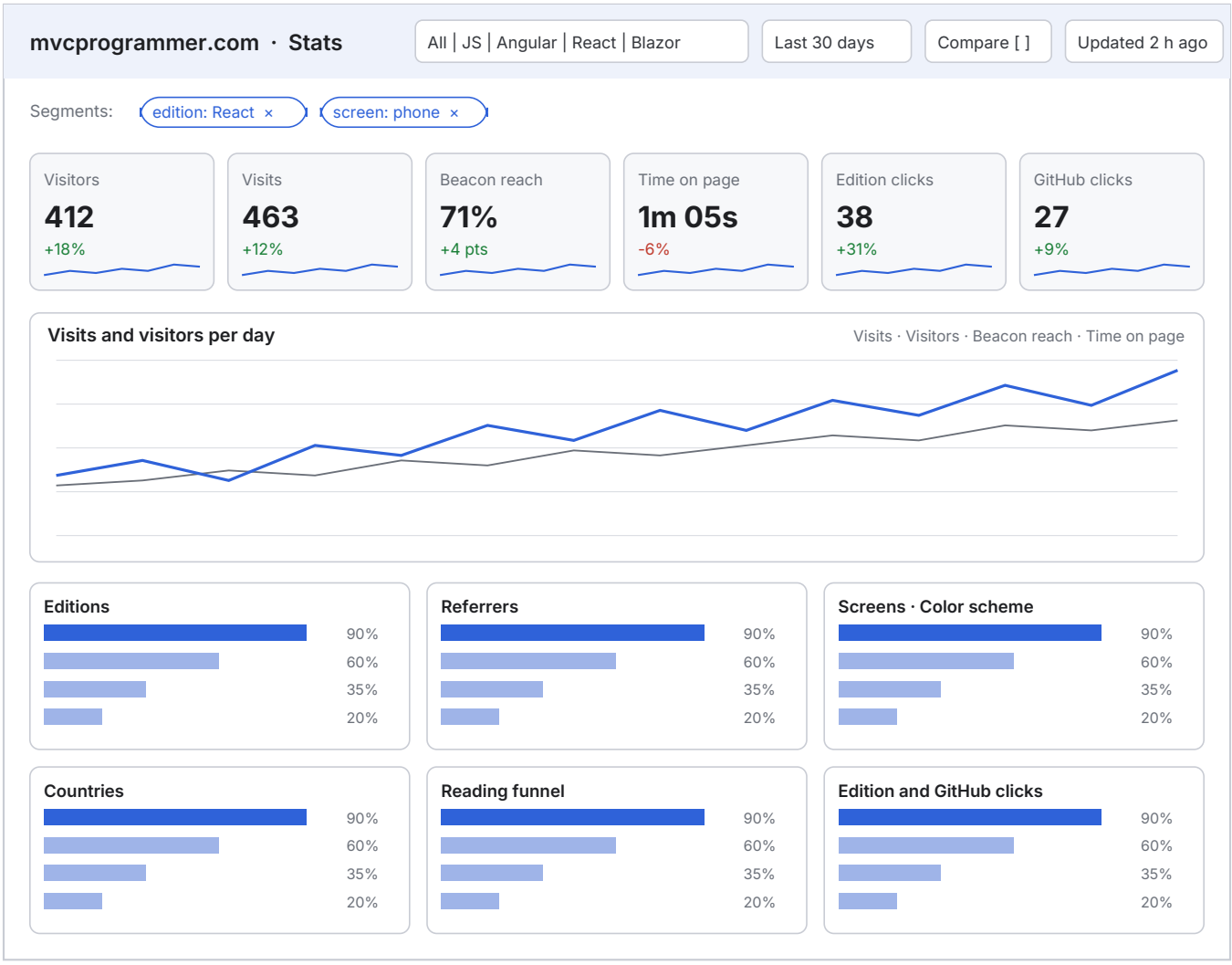
| Term | Definition |
|---|---|
| Request | One log line |
| Scanner | A request whose user agent matches the bot list, has no user agent, or ended in a 4xx; excluded from everything below except the scanner share |
| Visitor | One browser: the pair of client IP and user agent string, hashed with a secret salt and truncated; the hash lives only in the day files, never the IP |
| Visit | A visitor's requests with no gap over 30 minutes, which include a page and at least one asset, API call, or ping. This is the unit everything else counts |

| Term | Definition |
|---|---|
| Beacon reach | Visits that sent any ping (load, view, click, or leave), as a share of visits. Measures how many visits ran JavaScript at all; the rest are bounces before the script ran, or non-browsers that slipped past the bot list. Screen and color scheme come from the load ping alone, which shipped on October 18, 2026 |
| Engaged visit | A visit with at least one section view ping |
| Bounce rate | 1 minus engaged visits over beacon-reached visits. Defined only over visits that could have engaged |
| Time on page | The largest leave value a visit reported, in seconds; shown as median and 75th percentile, never as a mean alone |
| Section reach | Engaged visits that reached a section, as a share of engaged visits, in page order; the shape of the funnel is the point, not any one number |
| Edition click-through | Visits with a click ping of kind `edition`, by target edition and by place (header menu, About chips, hero) |
| GitHub click-through | Visits with a click of kind `github`, split into profile and repository, by place |
| Returning visitor | A visitor token seen on more than one day inside the selected range |
| Unique visitors over a range | The number of distinct visitor tokens among the visits in the range, under whatever filters are active |
| Compare | Every headline number against the same-length period immediately before the selected range; shown as a signed percentage with an arrow |

# 7. Dashboard design

## 7.1 Layout

The page follows the layout that Plausible, Fathom, and the GA4 overview share, because it is what people expect from an analytics page: a thin header with the controls, a row of headline tiles, one large chart, then a grid of breakdown panels. Everything below the header reacts to the controls.




The desktop layout. On a phone the tiles fall into two columns, the panels stack, and the controls fold into one Filters sheet.

## 7.2 Controls

- **Date range.** Presets Today, 7 days, 30 days, 90 days, This month, Last month, and a custom picker with two calendars. The default is 30 days. The chosen range lives in the URL (`?from=2026-09-18&to=2026-10-18`), so the browser's back button and bookmarks work and a view can be reopened later.
- **Compare.** A toggle that overlays the previous period on the chart as a dashed line and puts a delta on every tile.
- **Segments.** Clicking a row in any breakdown panel (an edition, a screen class, a referrer, a country) adds it as a chip in the header and narrows every panel to that segment, as Plausible does. Chips are removable one by one or all at once, and they live in the URL too.
- **Edition switcher.** The four editions plus "All", as a segmented control in the header, because it is the filter you will reach for most.
- **Updated stamp.** "Updated 2 h ago" from `index.json`, with a tooltip giving the exact time, so stale data is never mistaken for a quiet day.

### 7.3 Headline tiles

Six tiles, each with the number, a delta against the compare period, and a seven-day sparkline:

| Tile | Number | Note |
|---|---|---|
| Visitors | Unique visitors in range | The one to glance at first |
| Visits | Visit count | |
| Beacon reach | Share of visits that ran the script | Shown greyed when the range predates the load ping |
| Time on page | Median seconds visible, formatted as 1m 05s | 75th percentile in the tooltip |
| Edition clicks | Visits that switched edition | Percent of beacon-reached visits in the tooltip |
| GitHub clicks | Visits that opened the profile or a repository | Profile versus repository split in the tooltip |

Resume downloads appear as a seventh tile only when the range has any, so a zero never takes up a tile.

### 7.4 Main chart

Visits and visitors per day as two lines, the previous period dashed when Compare is on. A metric switcher above the chart swaps in beacon reach, time on page, or edition clicks. Hovering a day gives a tooltip with every series; clicking a day narrows the range to that day. Under 90 days the x axis is daily; from 90 days it switches to weekly bars, because daily lines past three months become noise.

### 7.5 Breakdown panels

| Panel | Chart | Interaction |
|---|---|---|
| Editions | Horizontal bars, four rows, share of visits | Click to segment |
| Referrers | Table: host, visits, share; direct and unknown grouped at the bottom | Click to segment |
| Screens and color scheme | Two small donuts side by side (three and two categories, the only place a donut is honest) | Click to segment |
| Countries | Table with a flag glyph, visits, share; derived from the edge location table, so labelled "served from" rather than "located in" | Click to segment |
| Reading funnel | Horizontal bars in page order, share of engaged visits that reached each section | Hover for counts |
| Time on page | Histogram over the six buckets | Hover for counts |
| Edition clicks | Stacked bars: target edition by place (menu, chips, hero) | Hover for counts |
| GitHub and profile clicks | Table: target (profile, each repository, LinkedIn, Flickr), visits, place | Hover for counts |
| Time of day | 24-column bar in the site's time zone | |
| Scanner share | A small line, visits versus scanner requests, for operations only | Collapsed by default |

### 7.6 Visual language

- The dashboard loads the site's shared `styles.css` for tokens, fonts, and the light and dark themes, with no palette override, so it wears the base JetBrains Rider look that none of the four editions uses any more. That makes it recognizably part of the family and visibly not one of the public faces.
- Chart colors come from the tokens: the accent for the primary series, the muted text color for the compare series, and a fixed six-color categorical ramp for the edition and place breakdowns, chosen for contrast in both themes and checked for the common color-vision deficiencies.
- Numbers are formatted the way the site already does it: thousands separators, one decimal on percentages, durations as minutes and seconds.
- Deltas are green up and red down, with an arrow glyph so color is never the only signal.
- Every chart has an accessible table twin behind a "Show as table" link, which also serves as the CSV export source.

### 7.7 States

- **Loading.** Tiles and panels render skeletons in the final layout, so nothing jumps.
- **Empty.** A range with no days says so, with the earliest and latest available days as links.
- **Partial.** Days without a file are shaded on the chart and listed in a note, rather than silently read as zero.
- **Error.** A failed fetch shows a retry button and the HTTP status, never a blank page.
- **Signed out.** Cognito handles it; the app only ever sees signed-in requests.

### 7.8 Responsive and accessible

- Phone: tiles in two columns, panels stacked, the chart full width with a shorter height, the controls collapsed into a single "Filters" sheet.
- Tablet: tiles in three columns, panels in two.
- Desktop: as in the wireframe.
- Every control is keyboard reachable; charts expose their table twin to screen readers; focus rings and reduced-motion follow the site's stylesheet.

## 8. Front end

### 8.1 Stack

- React 19, TypeScript strict, Vite, under `stats/`, mirroring the React edition's conventions: no CSS-in-JS, no `style` attributes, `bundles/` output, `cssTarget` pinned, Vitest with Testing Library.
- Chart.js 4 with `react-chartjs-2`, tree-shaken to the line, bar, and doughnut controllers.
- No date library. The Intl APIs cover formatting, and the day arithmetic is a dozen lines against ISO day keys.
- State: the URL is the single source of truth for range, compare, edition, and segments; a small reducer turns URL changes into a query; a loader fetches and caches day files; pure functions aggregate.

### 8.2 Structure

```
stats/
  index.html                shell, shared stylesheet link, boot screen
  src/stats.css             the additions: grid, tiles, chips, skeletons, chart frames
  src/main.tsx
  src/app/
    App.tsx                 header, controls, routed query from the URL
    pages/Dashboard.tsx
    components/             DateRangePicker, SegmentChips, EditionSwitcher, KpiTile, Sparkline,
                            TimeSeriesChart, BarsPanel, DonutPanel, TablePanel, FunnelPanel, Updated
    aggregation/            pure functions: sum days, union visitor tokens, compute deltas,
                            apply a segment, bucket histograms
    data/                   DayFile and Index types, loader with an in-memory cache, schema check
    formatters/             numbers, durations, percentages, day labels
    query/                  URL <-> query state
  src/testing/              fixtures: synthetic day files, a golden 30-day set
```

### 8.3 Performance budget

- First load under 150 KB compressed including Chart.js (the first build came in at 146 KB); the shared stylesheet and fonts are already cached from the site.
- A 30-day range is 30 small fetches in parallel through HTTP/2, cached by the browser for an hour and by CloudFront for a day; a 90-day range stays under a second on cellular.
- Aggregation is synchronous and under 20 ms for a year of days on a phone.

## 9. Data pipeline

### 9.1 Code

Split `tools/cf-stats.mjs` into `tools/lib/cf-logs.mjs` (download, parse, scanner filter, session building, the metric functions) and two thin callers: the existing `cf-stats.mjs` for the text report and a new `stats-export.mjs` that writes day files and the index. The split is the first task because it gives both tools one definition of every term and lets the exporter be unit-tested against fixture log lines.

The exporter takes `--days N` (default 3), rebuilds those days from the logs, writes `days/YYYY-MM-DD.json` and a refreshed `index.json` to a local folder, and the workflow syncs that folder to the bucket. The visitor salt comes from a repository secret so tokens are stable across runs but useless outside them.

### 9.2 Workflow

`.github/workflows/stats.yml`: cron at 00, 06, 12, and 18 UTC plus `workflow_dispatch`; checks out, sets up Node 24, assumes the deploy role, runs the exporter, syncs the output to `s3://mvcprogrammer.com/stats/data/` with a one-hour browser TTL, and invalidates `/data/*` on the stats distribution. A failed run emails you through GitHub's normal workflow notifications, which is the only alerting this needs.

The deploy role needs read on the log bucket; it already has write on the site bucket and invalidation on distributions, so the change is one IAM statement, documented in `docs/DEPLOY.md` beside the others.

### 9.3 The local path still works

`node tools/stats-export.mjs --days 365 --out stats/public/data` followed by `npm start` in `stats/` gives the full dashboard on your laptop from the cached logs, with no sign-in and no AWS beyond the log download. That is how it will be developed, and it is the fallback if you ever decide the hosted version is not worth keeping.

## 10. Hosting and security

- **Distribution.** `stats.mvcprogrammer.com` on a new CloudFront distribution with an ACM certificate in us-east-1, Origin Access Control to the `stats/` prefix of the site bucket, the S3 403 rewritten to `index.html` for deep links, and the same response headers policy as the site: same-origin CSP, two-year HSTS, `X-Frame-Options: DENY`, `nosniff`, a locked Permissions-Policy. `X-Robots-Tag: noindex` and a `robots.txt` disallow on top, since a sign-in page should not be in search results either.
- **Sign-in.** A Cognito user pool with a single user, hosted UI on a Cognito domain, passkey or TOTP as the second factor, and a Lambda@Edge viewer-request function using the `cognito-at-edge` library: no session cookie means a redirect to the hosted sign-in, a valid session means the request proceeds, and the function sets the cookies on return. Sessions last 30 days with refresh, and a `/signout` path clears them. Nothing in the repository holds a secret; Lambda@Edge has no environment variables, so the function reads the pool id, app client id, hosted UI domain, and a CSRF nonce secret from SSM Parameter Store once per container, and fails closed (503) if any is missing.
- **Data.** The day files sit under the same prefix, so the same edge check protects them. Because the prefix lives in the public site's bucket, which the apex distribution serves from its root, a bucket-policy statement denies `stats/*` to every distribution but the stats one; the export uploads nothing until that is in place. They hold no IP addresses, no user agents, and no path a visitor typed; visitor tokens are salted hashes that cannot be reversed or joined to anything else.
- **Logs.** Standard logging off for the stats distribution; there is nothing to learn from one person's visits to a private page.
- **Costs.** Cognito: free under 50,000 monthly users. Lambda@Edge: every request to the distribution, about fifty per page view with the day files, so a few thousand a month for one reader at $0.60 per million, rounding to zero. CloudFront and S3: kilobytes. GitHub Actions: free minutes on a public repository, about four minutes a day used. ACM: free. Estimated total: well under one dollar a month.

## 11. Build plan

Estimates are working days for one person, with Claude pairing. Each phase ends in a commit that leaves the site deployable.

| Phase | Deliverable | Acceptance | Estimate |
|---|---|---|---|
| 0. Shared parser | `tools/lib/cf-logs.mjs`, `cf-stats.mjs` unchanged in output, `stats-export.mjs` writing day files, unit tests on fixture log lines, a golden day file | The text report prints identically before and after; the golden file matches to the byte on repeat runs | 1 day |

| Phase | Deliverable | Acceptance | Estimate |
|---|---|---|---|
| 1. Dashboard, local | The `stats/` app reading day files from `public/data`: controls, tiles, main chart, all panels, light and dark, phone layout, tests | Every panel matches the text report for the same range; Lighthouse accessibility 100; bundle inside budget | 3 days |
| 2. Pipeline | `stats.yml` on the six-hour cron, IAM read on the log bucket, data synced to the bucket, `DEPLOY.md` section | Three consecutive scheduled runs green; `index.json` on S3 advancing | 0.5 day |
| 3. Hosting and sign-in | Distribution, certificate, Route 53, headers, Cognito pool and user, Lambda@Edge, the `deploy-stats` job in `deploy.yml` | Signed out: every path redirects to sign-in, including a data file URL; signed in: the dashboard loads on phone and laptop; passkey enrolled | 1.5 days |
| 4. Compare, segments, export | Previous-period overlay, segment chips from every panel, CSV export, URL state round-trips | A bookmarked URL with range, compare, and two segments reopens identically | 1.5 days |
| 5. Polish and write-up | Empty and partial states, skeletons, README for `stats/`, a guideline PDF in the series, profile README row | Reads like the other editions' documentation | 1 day |

Total: about eight and a half working days, which is two to three calendar weeks at the pace the editions were built. Phases 0 to 2 are useful on their own; the hosted version is Phase 3.

Later, if wanted: a weekly email digest from the same exporter (SES is already set up), a country map, contact form and resume alert records written into the data prefix by their Lambdas, and a "share this view" signed link that expires.

## 12. Testing and quality

- **Parser and exporter.** Unit tests over a fixture of real-shaped log lines covering every row type: a bot, a 4xx, a page with assets, a ping of each kind, an API call with a skill, a resume download, two visits by one visitor 31 minutes apart. A golden 30-day fixture guards the day-file schema; any intentional change bumps `schemaVersion`.
- **Aggregation.** Property tests on the pure functions: summing days is order-independent; a segment never exceeds its total; unions of visitor tokens never shrink.
- **Components.** Testing Library tests per component, following the React edition's patterns: the date picker emits the right query, chips add and remove, the tiles format deltas, charts expose their table twin.
- **Edge function.** Tests for no cookie, an expired token, a valid token, and the sign-out path, run against the library's test harness.
- **End to end, manual.** A checklist in the `stats/` README: sign in on phone and laptop, change every control, open a data file URL signed out, check both themes.
- **Standards.** The repository's coding standards apply as they do to the React edition: XML-style doc comments on exported functions, no inline styles, analyzers and lint as errors, tests beside sources.

## 13. Risks and mitigations

| Risk | Mitigation |
|---|---|
| Late log delivery leaves a day under-counted | Every run rebuilds the last three days; the "Updated" stamp and the partial-day shading make the rebuild window visible |
| Bot traffic slipping past the filter inflates visits | Beacon reach is a first-class metric precisely so that visits without a load ping are visible; the bot list is shared with the resume Lambda and grows from the scanner panel |
| Chart library drifts from the CSP | Chart.js is canvas-only; a CSP violation test in CI loads the built page under the site's policy and fails on any report |
| Lambda@Edge replication and us-east-1 constraints surprise a deploy | Documented step by step like the other distributions; the Basic auth fallback is a 30-line CloudFront Function if the Cognito path stalls |
| The six-hour cron is delayed by GitHub during busy periods | Accepted; the stamp shows it, and the manual run button exists |
| Scope creep toward a general analytics product | Non-goals are written down; anything not in section 7 is a Phase 6 conversation |

## 14. Decisions to confirm

1. **Hosted at `stats.mvcprogrammer.com`, or local-only?** Recommended: hosted, because the phone is where you will actually look.
2. **Cognito sign-in with a passkey, or Basic auth?** Recommended: Cognito. It is the standard answer and it is free; Basic auth is the fallback if you want it live this week.
3. **React, or Angular?** Recommended: React, for the charting ecosystem and the conventions already proven in the React edition. Angular would be equally at home and is a fair choice if you want the practice.
4. **Six-hourly refresh, or hourly?** Recommended: six-hourly. Hourly costs nothing extra and is a one-line change if the wait annoys you.
5. **Chart.js, or Apache ECharts?** Recommended: Chart.js for size and CSP fit. ECharts if you later want maps and richer interaction in one library.

**Confirmed October 18, 2026:** all five as recommended, so the dashboard is hosted at `stats.mvcprogrammer.com`, behind Cognito, built with React and Chart.js, refreshed every six hours. The work happens on the `feature/stats-dashboard` branch, one commit per phase or smaller, with a code review before the branch merges into `main` and deploys.

## Appendix A. Alternatives considered and set aside

- **Third-party analytics (Plausible, Fathom, Umami cloud).** Excellent products, and the dashboard design borrows from them openly. Set aside because the site's privacy page promises no third-party trackers, the data would leave your account, and the point of the exercise is to own it.
- **Self-hosted Umami or Matomo.** A database and a server, with patching, for one reader. Against the site's no-server rule.

- **Amazon QuickSight over Athena.** The "AWS-native" route: Athena queries the raw logs, QuickSight draws dashboards. QuickSight is about $24 a month for an author, Athena bills per query over the gzipped logs, and the result is a generic BI tool, not a page in the site's own voice.
- **CloudWatch dashboards.** The logs are in S3, not CloudWatch, and the beacon semantics (visits, reach, funnel) are not expressible as CloudWatch metrics without a Lambda that computes them anyway.
- **Rendering the dashboard as static HTML from the exporter.** Tempting for KISS: no JavaScript app at all. Set aside because the date picker, comparison, and segments are the features that make it a dashboard rather than a report, and they need a client.

## Appendix B. Panel specifications

| Panel | Data fields | Sort | Empty text |
|---|---|---|---|
| Editions | `byEdition` summed over the range | Fixed order: JS, Angular, React, Blazor | "No visits in this range" |
| Referrers | `byReferrer` summed, direct grouped | Visits descending, top 10, "Others" row | "All visits were direct" |
| Screens | `byScreen` summed over beacon-reached visits | Phone, tablet, desktop | "No beacon data in this range" |
| Color scheme | `byScheme` summed over beacon-reached visits | Light, dark | Same |
| Countries | `byCountry` summed | Visits descending, top 10 | "No visits in this range" |
| Reading funnel | `sectionReach` summed over engaged visits | Page order | "No section views in this range" |
| Time on page | `timeOnPage` buckets summed | Bucket order | "No time-on-page data in this range" |
| Edition clicks | `clicks` with kind `edition`, by target and place | Target in edition order; place stacked | "No edition clicks in this range" |
| GitHub and profile clicks | `clicks` with kind `github` or `social`, by target and place | Profile first, then repositories by count, then LinkedIn and Flickr | "No profile clicks in this range" |
| Time of day | `byHour` summed | Hour | "No visits in this range" |
| Scanner share | `requests`, `scannerRequests` per day | Day | Hidden when zero |

## Appendix C. A day file, abridged


```
{
  "schemaVersion": 1,
  "day": "2026-10-17",
  "timeZone": "America/New_York",
  "requests": 1854,
  "scannerRequests": 593,
  "visits": [
    { "token": "37653da2b1", "hour": 10, "editions": ["js", "angular"], "country": "US", "edge": "IAD",
      "referrer": "www.linkedin.com", "loads": 1, "screen": "desktop", "scheme": "dark",
      "sections": ["about", "portfolio"], "seconds": 75, "clicks": ["edition/angular/header"],
      "apiCalls": 1, "skills": ["Angular"] },
    { "token": "37653da2b1", "hour": 10, "editions": ["js"], "country": "US", "edge": "IAD",
      "loads": 1, "screen": "desktop", "scheme": "dark", "seconds": 5 },
    { "token": "aeb7077a18", "hour": 22, "editions": ["react"], "country": "DE", "edge": "FRA",
      "loads": 1, "screen": "phone", "scheme": "light", "seconds": 20,
      "clicks": ["github/profile/about"], "resumes": ["react"] },
    { "token": "aeb7077a18", "hour": 23, "editions": ["js"], "country": "DE", "edge": "FRA",
      "sections": ["about"] }
  ]
}
```